

Лабораторна робота №4.

Синхронізація завдань через семафори у FreeRTOS

Мета роботи. Ознайомитися з принципом роботи семафорів у FreeRTOS, навчитися використовувати бінарні, лічильні семафори та м'ютекси для синхронізації завдань і взаємодії між завданнями та перериваннями.

Теоретичні відомості.

FreeRTOS надає три основні типи семафорів:

- Бінарний семафор (Binary Semaphore) – використовується для сигналізації про події (1 або 0). Аналог прапорця «подія відбулася».
- Лічильний семафор (Counting Semaphore) – зберігає кількість подій, що сталися. Корисно, коли події можуть траплятися частіше, ніж завдання їх обробляє.
- М'ютекс (Mutex) – використовується для взаємного виключення доступу до ресурсу (наприклад, UART або SPI). Забезпечує, щоб тільки одне завдання одночасно використовувало ресурс.

Хід виконання роботи.

1. Створіть проєкт FreeRTOS у середовищі розробки VS Code.
2. Проаналізуйте даний код в якому реалізовано сигналізацію між кнопкою і світлодіодом через бінарний семафор і використано лічильний семафор для підрахунку кількості натискань.

1	#include <stdio.h>
2	#include "pico/stdlib.h"
3	#include "FreeRTOS.h"
4	#include "task.h"
5	#include "semphr.h"
6	
7	SemaphoreHandle_t xBinSemaphore;
8	SemaphoreHandle_t xCountSemaphore;
9	
10	void vTaskButton(void *pvParameters) {
11	while (1) {
12	if (gpio_get(15)) {
13	xSemaphoreGive(xBinSemaphore);
14	xSemaphoreGive(xCountSemaphore);

```

15     vTaskDelay(pdMS_TO_TICKS(300));
16 }
17 }
18 }
19
20 void vTaskLed(void *pvParameters) {
21     while (1) {
22         if (xSemaphoreTake(xBinSemaphore, portMAX_DELAY)) {
23             gpio_put(25, !gpio_get(25));
24             printf("LED toggled by binary semaphore\n");
25         }
26     }
27 }
28
29 void vTaskCounter(void *pvParameters) {
30     int count = 0;
31     while (1) {
32         if (xSemaphoreTake(xCountSemaphore, portMAX_DELAY)) {
33             count++;
34             printf("Button pressed %d times\n", count);
35         }
36     }
37 }
38
39 int main() {
40     stdio_init_all();
41     gpio_init(25); gpio_set_dir(25, true);
42     gpio_init(15); gpio_set_dir(15, false);
43
44     xBinSemaphore = xSemaphoreCreateBinary();
45     xCountSemaphore = xSemaphoreCreateCounting(10, 0);
46
47     xTaskCreate(vTaskButton, "Button", 256, NULL, 2, NULL);
48     xTaskCreate(vTaskLed, "LED", 256, NULL, 1, NULL);
49     xTaskCreate(vTaskCounter, "Counter", 256, NULL, 1, NULL);
50
51     vTaskStartScheduler();
52     while (1) {}
53 }

```

3. Модифікуйте код так, щоб він коректно працював на Pico Sensor Kit.
4. Створіть обробник переривання кнопки, функція **SemaphoreGiveFromISR()** - передає семафор з контексту переривання, **portYIELD_FROM_ISR()** - якщо

завдання, що очікує цей семафор, має вищий пріоритет, то планувальник одразу перемикається на нього, не чекаючи завершення поточного контексту.

```
1 void button_isr(uint gpio, uint32_t events) {
2     BaseType_t xHigherPriorityTaskWoken = pdFALSE;
3     xSemaphoreGiveFromISR(xButtonSemaphore, &xHigherPriorityTaskWoken);
4     portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
5 }
```

5. У функції `main()` встановлюємо переривання на кнопку, коли відбувається спадний фронт при натисканні:

```
1 gpio_set_irq_enabled_with_callback(15, GPIO_IRQ_EDGE_FALL, true,
  &button_isr);
```

6. Модифікуйте приклад коду вище, так, щоб стан кнопки зчитувався тільки через переривання.
7. Створіть два завдання, які надсилають дані через UART. Використайте **`xSemaphoreCreateMutex()`**, щоб уникнути одночасного доступу до UART.

```
1 #include "FreeRTOS.h"
2 #include "task.h"
3 #include "semphr.h"
4 #include "pico/stdlib.h"
5
6 // Дескриптор м'ютекса
7 SemaphoreHandle_t xUartMutex;
8
9 // Завдання 1 — друкує повідомлення Task 1
10 void Task1(void *pvParameters) {
11     while (1) {
12         if (xSemaphoreTake(xUartMutex, portMAX_DELAY) == pdTRUE) {
13             printf("Task 1: Hello from Task 1!\r\n");
14             vTaskDelay(pdMS_TO_TICKS(200)); // імітація роботи з UART
15             printf("Task 1: Finished writing.\r\n");
16             xSemaphoreGive(xUartMutex); // звільняємо ресурс
17         }
18         vTaskDelay(pdMS_TO_TICKS(1000));
19     }
20 }
21
22 // Завдання 2 — друкує повідомлення Task 2
```

```

23 void Task2(void *pvParameters) {
24     while (1) {
25         if (xSemaphoreTake(xUartMutex, portMAX_DELAY) == pdTRUE) {
26             printf("Task 2: Sending data to UART...\r\n");
27             vTaskDelay(pdMS_TO_TICKS(300)); // імітація тривалої операції
28             printf("Task 2: Done.\r\n");
29             xSemaphoreGive(xUartMutex);
30         }
31         vTaskDelay(pdMS_TO_TICKS(1200));
32     }
33 }
34
35 int main() {
36     stdio_init_all();
37
38     // Створюємо м'ютекс
39     xUartMutex = xSemaphoreCreateMutex();
40
41     if (xUartMutex == NULL) {
42         printf("Failed to create mutex!\r\n");
43         while (1);
44     }
45
46     // Створюємо два завдання
47     xTaskCreate(Task1, "Task1", 512, NULL, 1, NULL);
48     xTaskCreate(Task2, "Task2", 512, NULL, 1, NULL);
49
50     // Запускаємо планувальник
51     vTaskStartScheduler();
52
53     while (true);
53 }

```

8. Проаналізуйте код.

9. Для демонстрації колізії вимкніть м'ютекс (потрібно закоментувати xSemaphoreTake() / xSemaphoreGive()) і порівняйте результати.

10. Додайте третє завдання для ще більшої конкуренції.